시퀀스 모델

RNN 은닉 상태와 순전파 · BPTT와 numpy RNN 언어모델 · 그래디언트 소실과 LSTM/GRU

Machine Learning 1 · 11주차

한국과학영재학교 (KSA)

Week 11

이번 주차 개요

이 주차를 마치면

- RNN의 순전파 수식(은닉 상태 갱신식)을 손으로 유도하고, 파라미터 수가 시퀀스 길이에 무관한 이유를 설명
- BPTT의 재귀 구조(본 시점 오차 + 다음 시점에서 흘러온 그래디언트의 합)를 유도하고, numpy로 문자 단위 RNN 언어모델을 학습시킨다
- 그래디언트 곱적이 W_{hh} 고유값 1을 기준으로 지수적으로 소실·폭발함을 숫자로 확인하고, LSTM/GRU가 "곱은 조절 스위치, 덧셈은 통로"의 ResNet 스킵 연결 시간 축 버전임을 원리 수준으로 서술
- 기본 RNN → GRU → LSTM → Attention 사이에서 언제 무엇을 고를지 판단하고, 게이트가 소실을 "완화"할 뿐 "해결"하지 않는 이유를 설명

- **11.1 RNN: 은닉 상태와 순전파** — "은닉 상태는 지금까지 읽은 것의 요약"이라는 핵심 아이디어를 순전파 수식으로, 스칼라 RNN의 3개 시점과 고정점 수렴을 손으로 계산
- **11.2 BPTT와 numpy RNN 언어모델 실습** — BPTT를 시점 재귀식으로 유도하고, "abcabc…"를 배우는 문자 모델로 LLM의 "다음 토큰 예측"을 가장 축소된 형태로 재현
- **11.3 그래디언트 소실의 재발과 LSTM/GRU** — 소실·폭발을 숫자로 확인하고, 게이트(덧셈 통로)가 기억을 살리는 "기억 과제" 실험으로 마무리

이 장을 관통하는 하나의 실: "기억과 그 감쇠" — 11.1에서 감쇠하며 갱신되는 요약, 11.2에서 그 감쇠가 그래디언트 소실이라는 이름으로 재등장, 11.3에서 게이트가 감쇠를 늦추되 순차성이라는 근본 한계는 남긴다.

11.1 RNN: 은닉 상태와 순전파

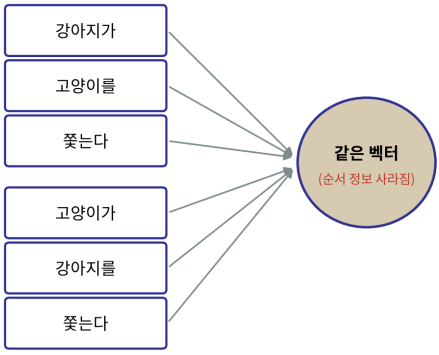
왜 순서가 중요한가

- MLP·CNN에 이 문장을 넣으려면 단어를 **벡터 하나로 뭉쳐야** 한다 → **순서 정보가 사라짐**
- 시퀀스의 "왼쪽→오른쪽"은 **시간의 방향성**: 앞 단어는 뒤 단어를 영향주지만 뒤 단어는 앞 단어를 알 수 없다

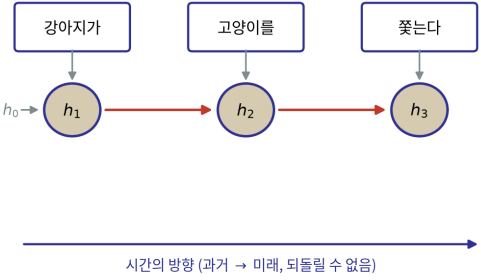
이 비대칭을 모델이 반영할 수 있어야 한다 — RNN의 출발 질문.

그림으로: 순서를 다루는 두 가지 방법

MLP / CNN: 단어를 벡터 하나로 뭉침



RNN: 시간 순서대로 하나씩 처리



MLP/CNN은 단어들을 하나의 벡터로 뭉쳐 순서 정보가 사라지지만, RNN은 $h_0 \to h_1 \to h_2 \to \cdots$ 처럼 시간 순서대로 하나씩 처리해 "과거 $\to$ 미래"라는 방향성을 구조 자체에 담는다.

역사: Elman 네트워크와 “시간 속의 구조”

- 구조: 은닉층을 **센트럴 유닛**(지금 입력 처리)과 **라테럴 유닛**(이전 상태의 복사본 저장)으로 나눔
- 라테럴 유닛의 출력을 **다음 시점의 입력으로** 다시 돌린다 — 지금의 RNN과 거의 똑같음
- 단순한 반복 연결을 “시간을 기억하는 장치”로 재해석한 것이 핵심

유명한 실험과 계보

같은 어미 “-pits”가 “pit(구덩이)”의 복수인지 “pitch(공)”의 복수인지를 문맥이 구분하는 듯이 행동한다는 관찰 (나중에 비판도 받았으나 “같은 토큰도 맥락에 따라 다른 내적 표현” 직관의 원형). 1997년 호흐하이터·슈미트허버가 제안한 **LSTM** 이 2010년대 음성인식·번역에서 비약적 성능을 보여주며 RNN이 비로소 주류가 됨 (11.3절에서 다시 만남).



Jeffrey Elman (1948–2018) — UC San Diego 인지과학과 창립 멤버
“Finding Structure in Time” (Cognitive Science, 1990) — RNN(Elman network)의 원 논문 저자

은닉 상태: 순차적 압축으로 요약

은닉 상태: 지금까지 읽은 내용의 요약

- h_t**는 문장 전체를** h**차원 벡터로 압축**
- **압축은 순간적이 아니라 순차적**

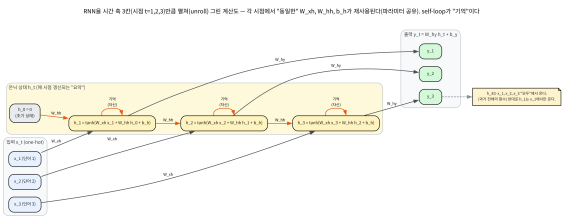
RNN의 순전파

시점 t마다 입력 x_t와 이전 은닉 상태 h_{t-1}을 받아, 새 은닉 상태 h_t와 (필요하면) 출력 y_t를 낸다:

$$h_t = \tanh(W_{xh}\, x_t + W_{hh}\, h_{t-1} + b_h), \qquad y_t = W_{hy}\, h_t + b_y$$

- **핵심**: W_{xh}, W_{hh}, W_{hy}가 **모든 시점에서 동일한 가중치를 재사용** (파라미터 공유)

시간을 펼쳐서(unroll) 그리기



h_t의 self-loop = “기억”의 시각화. $h_0 = 0$에서 시작.

시퀀스 전체를 한눈에 보는 가장 좋은 방법:
시간을 펼쳐서(unroll) 그리는 것.

- **같은 네트워크**가 시점마다 반복되어 있는 것이지, 각 시점에 **별개의 층**이 있는 것이 아님
- 출력 y_t는 $x_1, x_2, \ldots, x_t$ **전부**에서 온다 — y_3는 h_3를 통해 x_1, x_2의 영향도 받음
- 반대로 y_1은 x_1에서만 온다

“과거만 보고 미래는 못 본다”는 시간의 비가역성이 이 도식 안에 정확히 들어 있다.

파라미터 수: 시퀀스 길이와 무관

매개변수	크기	개수
W_{xh} (입력 $\to$ 은닉)	$H \times V$	$8 \times 26 = 208$
W_{hh} (은닉 $\to$ 은닉)	$H \times H$	$8 \times 8 = 64$
W_{hy} (은닉 $\to$ 출력)	$V \times H$	$26 \times 8 = 208$
b_h	H	8
b_y	V	26
합계		**514**

- 10글자든 1000글자든 **514개** — 파라미터가 시퀀스 길이에 비례하지 않는 것이 파라미터 공유의 직접적 혜택

코드: 한 걸음(rnn_step)과 전체($rnn_forward$)

```python
def rnn_step(x_t, h_prev, Wxh, Whh, b_h):
    z = [sum(Wxh[i][j]*x_t[j] for j in range(len(x_t))) +
         sum(Whh[i][j]*h_prev[j] for j in range(len(h_prev))) + b_h[i]
         for i in range(len(h_prev))]
    return [tanh(v) for v in z]

def rnn_forward(inputs, h0, Wxh, Whh, b_h):
    h = h0
    hidden_states = []
    for x_t in inputs:
        h = rnn_step(x_t, h, Wxh, Whh, b_h)
        hidden_states.append(h)
    return hidden_states
```

- 한 줄로 요약: **"같은 rnn_step을 T번 순서대로 반복하는 것"**

“무엇을” 담느냐: 누적합 예제

- 스칼라 RNN, $w_{xh} = 0.1$, $w_{hh} = 0.9$, $b_h = 0$, 입력 $[2, 3, 1, 4]$
- pre-activation z_t(tanh 전)를 따라가면:

$$z_1 = 0.200, \qquad z_2 = 0.480, \qquad z_3 = 0.532, \qquad z_4 = 0.879$$

기억의 감쇠 구조

$z_t = 0.1\,x_t + 0.9\,z_{t-1}$에 z_{t-1}을 재귀적으로 대입하면:

$$z_t = 0.1\,x_t + 0.1 \cdot 0.9\,x_{t-1} + 0.1 \cdot 0.9^2\,x_{t-2} + \cdots$$

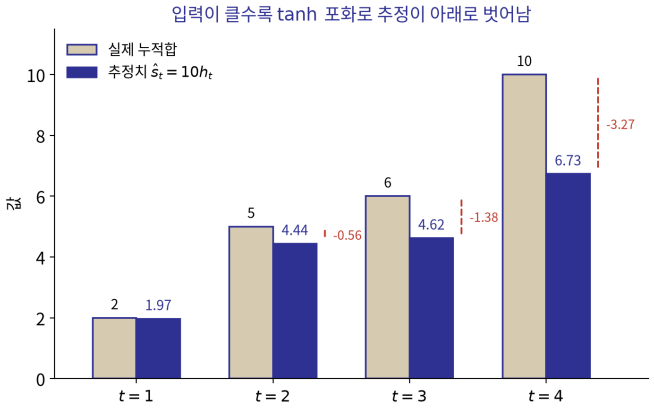
- 그래도 **감쇠의 구조**는 정확히 보인다: z_t는 과거 입력들의 **기하급수적으로 감쇠된 가중합**
- 0.9^k = k칸 전 입력의 "**기억 세기**"
- $k = 10$: $0.9^{10} \approx 0.35$ (10단어 전 정보의 35% 잔존), $k = 20$: ≈ 0.12

tanh를 거친 은닉 상태: 포화

실제로 tanh를 거치면 $h_t = \tanh(z_t) = [0.197,\ 0.444,\ 0.462,\ 0.673]$.

- 이 h_t를 누적합 추정치로 ($\hat{s}_t = 10 \cdot h_t$): 누적합 $[2, 5, 6, 10]$의 추정 $\rightarrow$ $[1.97,\ 4.44,\ 4.62,\ 6.73]$
- **입력이 클수록 포화로 아래로 벗어남**

그림으로: tanh 포화가 만드는 추정 오차



누적합 $[2, 5, 6, 10]$을 $\hat{s}_t = 10h_t$로 추정하면 $[1.97, 4.44, 4.62, 6.73]$ — 입력이 커질수록 tanh가 포화되어 실제값 아래로 벗어나는 간격이 벌어진다.

손으로 한 번: 스칼라 RNN 세 스텝

$$h_1 = \tanh(0.5 \times 1.0 + 0.8 \times 0) = \tanh(0.5) \approx 0.462$$

$$h_2 = \tanh(0.5 \times 1.0 + 0.8 \times 0.462) = \tanh(0.870) \approx 0.701$$

$$h_3 = \tanh(0.5 \times 1.0 + 0.8 \times 0.701) = \tanh(1.061) \approx 0.786$$

- 같은 입력($x_t = 1.0$)이 반복되는데도 h_t가 매 시점 다른 값으로 갱신 (0.462 $\rightarrow$ 0.701 $\rightarrow$ 0.786)

고정점과 그 “속도”

t	1	2	3	4	5	6	8	10
$z_t = 0.5 + 0.8\, h_{t-1}$	0.500	0.870	1.061	1.129	1.149	1.154	1.156	1.156
$h_t = \tanh(z_t)$	0.462	0.701	0.786	0.811	0.817	0.819	0.820	0.820

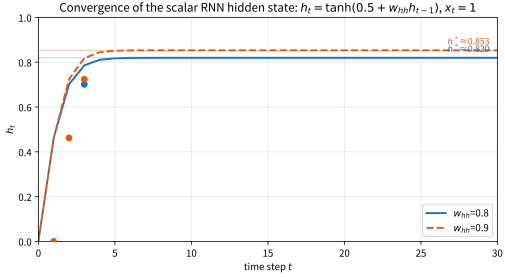
- h_t**는 고정점** $h^* \approx 0.8196$**으로 수렴**
- **수렴 속도는 증폭 인자로 결정**

핵심: 이 “증폭 인자”가 11.3절의 그래디언트 소실과 정확히 같은 수식

- **순전파**: h_{t-1}의 작은 변화가 h_t에 얼마나 전달되는지 재는 $\tanh'(z_t) \cdot w_{hh}$의 곱
- **역전파**: 손실의 그래디언트가 시점을 거슬러 올라갈 때 곱해지는 인자 $\tanh'(z_t) \cdot W_{hh}$ (11.2절)
- **구조적으로 동일: 같은 연쇄법칙 곱의 두 얼굴**

w_{hh}를 0.9로 키우면? — 그리고 발산

- 고정점은 $h^* \approx 0.853$으로 조금 높아짐
- 증폭 인자 $0.9 \times (1 - 0.853^2) \approx 0.245$ — 0.8 경우(0.263)보다 오히려 약간 **작아짐**
- 둘 다 6단계 만에 고정점의 10^{-3} 이내로 **정착**
- "**발산**"은 여전히 하지 않는다: 증폭 인자가 1보다 작기 때문
- 스칼라 RNN은 tanh의 범위 $[-1, 1]$에 갇혀 있어 발산 자체가 **불가능**



$w_{hh} = 0.8$: 0.820으로, $w_{hh} = 0.9$: 0.853으로 수렴 — 약 6단계 만에 정착.

단, 은닉 크기가 큰($H > 1$) RNN에서는 W_{hh}의 고유값이 1보다 크면 h_t가 실제로 발산하거나 진동할 수 있다 — 11.2절 "자주 하는 실수"의 그래디언트 폭발과 같은 뿌리 (Pascanu et al., 2013).

자주 하는 실수 1: “h_t는 x_t만 의존한다”

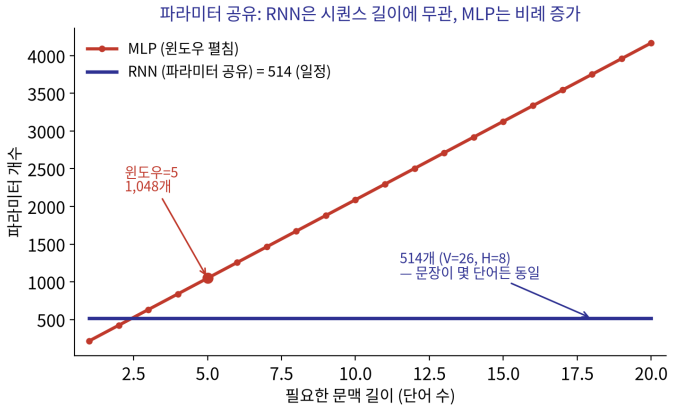
- h_t는 **과거 전체** $(x_1, \ldots, x_t)$**의 함수**이며, x_t 하나만 의존하지 않는다
- $W_{hh} = 0$으로 두면: $h_t = \tanh(W_{xh}\, x_t + b_h)$로 **“시점을 독립적으로 처리하는 MLP”로 퇴화**

$W_{hh}\, h_{t-1}$**을 빼면 RNN은 RNN이 아니다.**

자주 하는 실수 2: “시점마다 가중치가 달라진다”

- 아니다 — **세 행렬** W_{xh}, W_{hh}, W_{hy}**가 시점 전체에서 공유**된다 (파라미터 공유)
  1. 시퀀스 길이에 관계없이 **파라미터 수가 일정**
  2. “5번째 단어를 처리하는 능력”이 “150번째 단어를 처리하는 능력”에도 **자동으로 전이**

그림으로: 파라미터 공유가 만드는 차이



$V=26$, $H=8$ 문자 모델 기준: RNN은 문장이 몇 단어든 파라미터 **514개로 고정**, 반면 MLP는 필요한 문맥 길이(윈도우)에 **비례해 선형으로 증가**한다.

실습: 숫자 하나씩 코드로

3단계 손계산 · 고정점 · 누적합 예제를 하나의 코드로:

```python
import math

def rnn_step(x, h, wxh, whh, b=0.0):
    return math.tanh(wxh * x + whh * h + b)

# 단계3 손계산 (w_xh=0.5, w_hh=0.8, x=1, h0=0)
h = 0.0
for _ in range(3):
    h = rnn_step(1.0, h, 0.5, 0.8)
    print(f"h = {h:.4f}")       # 0.4621 / 0.7012 / 0.7860

# 고정점반복대입  : h = tanh(0.5 + 0.8h)
h = 0.0
for _ in range(100):
    h = rnn_step(1.0, h, 0.5, 0.8)
print(f"고정점 h* ~ {h:.4f}")   # 0.8196

# 누적합예제  (w_xh=0.1, w_hh=0.9), 입력 [2,3,1,4]
h = 0.0
for x in [2.0, 3.0, 1.0, 4.0]:
    h = rnn_step(x, h, 0.1, 0.9)
    print(f"h = {h:.4f}")       # 0.1974 / 0.4444 / 0.4621 / 0.6728
```

완전 버전(수렴 곡선 + unroll 다이어그램 포함)은 실습 노트북에서 실행.

핵심 공식 한눈에

항목	공식	무엇을 말하나
순전파 (재귀)	$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$	시점마다 은닉 상태 갱신
출력	$y_t = W_{hy}h_t + b_y$	시점별 예측 (선형, 활성함수 없음)
파라미터	$(HV+1) + H^2 + (VH+1)$	**시퀀스 길이 T와 무관**
기억의 감쇠	$z_t \approx \sum_k w_{hh}^{k-1} \cdot w_{xh}\, x_{t-k+1}$	w_{hh}가 “기억의 길이”를 결정

한 줄

RNN = **같은 세 행렬을 T번 순서대로 돌리며**, 이전 요약을 tanh로 압축해 갱신하는 구조.

FAQ: 은닉 상태 크기와 인코더-디코더

Q. 은닉 상태의 크기는 어떻게 정하나요?

"지금까지의 맥락을 얼마나 **풍부하게 요약**할 수 있는가"를 결정하는 하이퍼파라미터 — Week 9의 은닉층 뉴런 수와 같은 종류의 선택.

- 너무 작으면 **긴 문맥을 다 담지 못**하고
- 너무 크면 파라미터 증가 → **과적합 · 계산비용** 문제

Q. 입력과 출력이 다른 경우(번역)는?

"입력을 끝까지 읽고 **마지막 은닉 상태 하나**로부터 새 시퀀스를 순차 생성"하는 **인코더-디코더(encoder-decoder)** 구조를 쓰면 가능 — 단, 긴 입력을 하나의 벡터로 압축하면 **정보 손실**이 생김. **Week 12의 Attention이 정확히 이 한계를 보완하기 위해 출발했다** (Bahdanau et al., 2014).

FAQ: 문장 전체를 펼쳐 MLP에 넣으면 안 되는가

1. 파라미터: T에 비례해 팽창
2. 길이 가변성: 패딩 필요
3. 위치 인코딩 불필요

FAQ: 왜 tanh인가, h_0는 항상 0인가

Q. 왜 tanh인가? 시그모이드($0 \sim 1$)도 되는데

1. **중심화**: tanh 출력 범위 $[-1, 1]$은 0 중심 대칭. 시그모이드 $[0, 1]$은 항상 양수 $\rightarrow$ 여러 성분이 "동일 방향"으로 움직이는 경향, 학습이 느려짐 (9.2절의 시그모이드 "비대칭 골짜기")
2. **미분의 크기**: $\tanh'$ 최댓값 1 = 시그모이드 미분 최댓값 0.25의 **4배** — 같은 깊이에서 그래디언트가 4배 강하게 전달

반대로 **출력층** $y_t = W_{hy} h_t + b_y$는 활성함수 없이 **선형**: "다음 문자의 확률 분포(softmax)"나 "실수값(회귀)" 으로 해석되어야 하므로 $[-1, 1]$로 자르는 tanh가 오히려 방해 — 은닉층과 출력층은 **역할**이 다르다.

Q. h_0는 항상 0이어야 하나요?

꼭 아니다 — "시퀀스를 읽기 **전**에 알고 있는 것"을 담을 수 있는 초기 조건: (a) 0으로 고정 (가장 흔함), (b) **학습 가능** 파라미터 벡터, (c) **이전 시퀀스의 최종 은닉 상태**를 이어받아 이어지는 시퀀스 처리 (음성인식: 전 문장의 마지막 상태를 다음 문장의 초기 상태로).

11.2 BPTT와 numpy RNN 언어모델 실습

BPTT: 시간을 펼쳐서 역전파하기

- **시간 축을 따라 역전파 적용**

BPTT(Backpropagation Through Time)

시퀀스 길이 T만큼 신경망을 **펼쳐서(unroll)** 각 시점을 독립된 층처럼 취급한 뒤 Week 9의 역전파를 **그대로** 적용.

$$\frac{\partial h_T}{\partial h_1} = \prod_{t=2}^{T} \frac{\partial h_t}{\partial h_{t-1}} = \prod_{t=2}^{T} \text{diag}\left(tanh'(z_t)\right) W_{hh}$$

$z_t = W_{xh}x_t + W_{hh}h_{t-1} + b_h$, $\tanh'(z_t) = 1 - h_t^2$.

곱적: 소실·폭발의 씨앗

이 식이 말해주는 것: **먼 시점과 가까운 시점 사이를 건너는 그래디언트는 W_{hh}를 $T-1$번 곱하는 곱적**이 된다.

W_{hh} 고유값 < 1

곱이 **지수적으로 0**으로 — "그래디언트 **소실**"

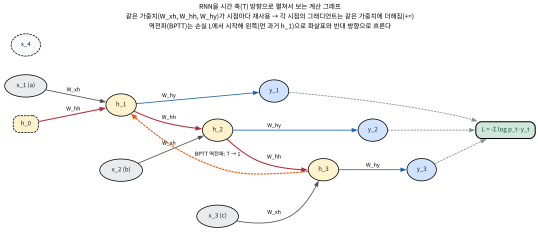
W_{hh} 고유값 > 1

곱이 **지수적으로** ∞로 발산 — "그래디언트 **폭발**"

11.3절에서 "그래디언트 소실·폭발"이라는 이름으로 다시 만나는 바로 그 문제 (Pascanu et al., 2013).

이번 절: 그 양보다 먼저, 이 곱적 구조를 정확히 따라가며 각 파라미터의 그래디언트가 어떻게 누적되는지 유도한다.

펼쳐진 계산 그래프: 시점이 곧 "층"



역전파는 오른쪽(손실)에서 왼쪽(과거)으로.

중요한 것 세 가지:

1. **수직 화살표(W_{hh})**: 자기-루프 ($h_{t-1} \to h_t$). 역전파에서는 **위에서 아래로** — h_t에서 h_{t-1}으로 거슬러 올라감
2. **각 시점의 출력 화살표(W_{hy})**: $h_t \to y_t$. 손실이 **모든 시점**의 y_t에서 나므로 역전파는 이 화살표들에서 시작
3. **같은 색 박스가 모든 시점에서 반복**: 시점 1과 시점 2의 W_{xh}, W_{hh}, W_{hy}는 **동일한** 가중치 (파라미터 공유) $\Rightarrow$ 각 시점의 그래디언트는 **같은 가중치의 그래디언트에 더해짐**

이 "누적"이 BPTT에서 가장 실수하기 쉬운 지점.

BPTT 유도: 시점마다 “(A) + (B)”

$$L = -\sum_{t=1}^{T}\sum_{k} y_{t,k}^{\text{true}} \log p_{t,k}$$

$p_t = \text{softmax}(W_{hy}h_t + b_y)$ = 시점 t의 다음-문자 분포, y_t^{true} = 원-핫 정답.

역전파는 T에서 1로 순차 진행

$$\underbrace{W_{hy}^{\top}\left(p_t - y_t^{\text{true}}\right)}_{(A)\text{ 본 시점의 출력 오차}} + \underbrace{W_{hh}^{\top}\left[\frac{\partial L}{\partial h_{t+1}} \odot (1 - h_{t+1}^2)\right]}_{(B)\text{ 시점 } t+1\text{에서 흘러온 그래디언트}}$$

Week 9의 2층 역전파와 같은 형태

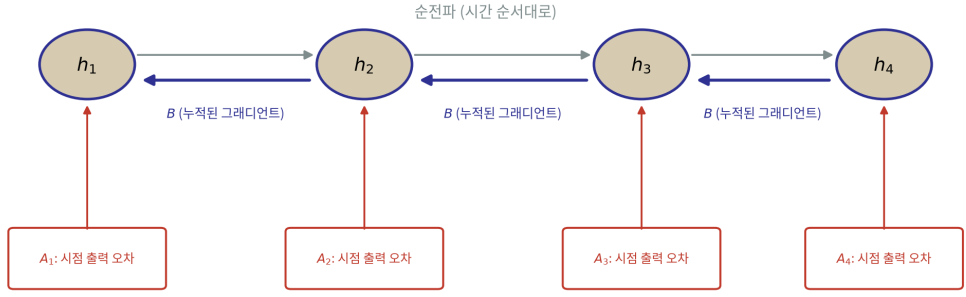
- **각 시점 역전파는 2층 신경망 형태와 같다**
- **재귀 구조: 사슬이 한 시점씩 이어진다**

핵심 문장

본 시점 오차 (A)에, **더 새로운 시점**에서 흘러온 그래디언트 (B)를 **더하고**, W_{hh}가 공유되므로 그 기여는 **같은 텐서에 누적**$(+=)$된다.

그림으로: BPTT의 재귀 구조

$$\frac{\partial L}{\partial h_t} = A_t + B_t \quad (A_t = \text{본 시점 오차}, B_t = \text{다음 시점에서 흘러온 것})$$



역전파는 미래(h_4)에서 과거(h_1)로 흐른다: 각 시점에서 본 시점의 출력 오차 A_t에 다음 시점에서 넘어온 그래디언트 B_t를 더해 W_{hh}가 공유되는 같은 텐서에 누적한다.

BPTT 코드: 핵심 루프

```
# 역전파: t = T-1, T-2, ..., 0 순으로 (0-indexed)
dh_next = np.zeros(H)                  # dL/dh_{t+1} 첫( 루프: 0)
dWxh = dWhh = dWhy = 0; dbh = dby = 0
for t in reversed(range(T)):
    p = softmax(Why @ hs[t] + by)
    dp = p.copy(); dp[targets[t]] -= 1  # p_t - onehot
    dWhy += np.outer(dp, hs[t]); dby += dp          # (A)
    dh = Why.T @ dp + dh_next                 # (A)+(B)의 합
    dz = dh * (1 - hs[t]**2)                  # * tanh'(z_t)
    dh_next = Whh.T @ dz                      # -> 이전시점로  B 전달
    dWhh += np.outer(dz, hps[t]); dbh += dz
    dWxh += np.outer(dz, one_hot(inputs[t]))
```

- **첫째** `dh = Why.T @ dp + dh_next`: 본 시점 오차에 **더 새로운 시점**에서 흘러온 그래디언트를 더하는 $(A) + (B)$
- **dWhh += ... (W_{hh} 공유)**

왜 p - onehot인가: softmax + CE

$$\frac{\partial L}{\partial z_j} = p_j - \mathbb{1}[j = k]$$

"softmax의 출력에서 **정답 클래스 자리에만 1을 빼고** 나머지는 그대로".

- 왜 단순해지는가: 연쇄법칙으로 펼치면 $\sum_k (-y_k/p_k)\, p_k(\delta_{kj} - p_j)$
  $= -\underbrace{\sum_k y_k \delta_{kj}}_{y_j} + p_j \underbrace{\sum_k y_k}_{1} = p_j - y_j$

손으로 한 번: 2시점 RNN의 순전파

$$W_{xh} = \begin{bmatrix} 0.3 & -0.2 & 0.0 \\ 0.5 & 0.1 & -0.3 \end{bmatrix}, \quad W_{hh} = \begin{bmatrix} 0.4 & 0.2 \\ -0.3 & 0.5 \end{bmatrix}, \quad W_{hy} = \begin{bmatrix} 0.6 & -0.4 \\ 0.1 & 0.5 \\ -0.2 & 0.3 \end{bmatrix}$$

- **시점 1** (입력 a = $e_a = [1,0,0]^\top$):

$$h_1 = \tanh(W_{xh} e_a) = \tanh \begin{pmatrix} 0.3 \\ 0.5 \end{pmatrix} = \begin{pmatrix} 0.2913 \\ 0.4621 \end{pmatrix}$$

- **시점 2** (입력 b = $[0,1,0]^\top$): $z_2 = W_{xh} e_b + W_{hh} h_1 = \begin{pmatrix} 0.0089 \\ 0.2437 \end{pmatrix}$,

  $h_2 = \tanh(z_2) = \begin{pmatrix} 0.0089 \\ 0.2390 \end{pmatrix}$

2시점 예제: 출력 분포와 손실

각 시점의 출력 분포 (softmax):

$$p_1 = \text{softmax}(W_{hy}h_1) = \begin{pmatrix} 0.2937 \\ 0.3848 \\ 0.3215 \end{pmatrix}, \qquad p_2 = \text{softmax}(W_{hy}h_2) = \begin{pmatrix} 0.2934 \\ 0.3622 \\ 0.3444 \end{pmatrix}$$

손실 (다음-문자 크로스엔트로피; 시점 1 목표 b(인덱스 1), 시점 2 목표 c(인덱스 2)):

$$L = -\log p_1[b] - \log p_2[c] = -\log 0.3848 - \log 0.3444 = 0.9550 + 1.0660 = \boxed{2.0210}$$

역전파는 시점 $2 \to 1$ 순서로

2시점 예제: 최종 그래디언트

$$\frac{\partial L}{\partial W_{hy}} = \begin{bmatrix} 0.0882 & 0.2058 \\ -0.1760 & -0.1977 \\ 0.0878 & -0.0081 \end{bmatrix}, \qquad \frac{\partial L}{\partial W_{hh}} = \begin{bmatrix} 0.1000 & 0.1587 \\ -0.0365 & -0.0579 \end{bmatrix}$$

$$\frac{\partial L}{\partial W_{xh}} = \begin{bmatrix} 0.2062 & 0.3434 & 0 \\ -0.2537 & -0.1254 & 0 \end{bmatrix}, \quad \frac{\partial L}{\partial b_h} = \begin{pmatrix} 0.5496 \\ -0.3791 \end{pmatrix}, \quad \frac{\partial L}{\partial b_y} = \begin{pmatrix} 0.5871 \\ -0.2530 \\ -0.3341 \end{pmatrix}$$

- **numpy 코드와 수치 미분 일치**
- W_{hh} **그래디언트 공유**

곱적의 양: 소실·폭발이 여기서 시작된다

유도에서 $\frac{\partial L}{\partial h_t}$가 시점을 거슬러 내려올 때마다 매번 $W_{hh}^{\top}$과 $(1-h^2)$ (즉 tanh′)를 곱받는다.

- 이 곱이 T번 반복 $\Rightarrow \prod_t tanh'(z_t)\, W_{hh}$

실습: numpy로 문자 단위 RNN 언어모델

"abcabcabcabc"(12문자)를 **문자 단위**로 예측하는 아주 작은 RNN 언어모델을 numpy로 직접 학습.

- **설정**: 어휘 {a, b, c} ($V = 3$), 은닉 크기 8, 다음-문자 예측 대상 **11개** ($T = 11$)
- **초기화**: 시드 42, $\mathcal{N}(0, 0.1^2)$ scale 0.1, 학습률 0.5, 500 에폭, 그래디언트 클리핑 5
- 입력 = 각 시점의 문자, 목표 = "다음 문자는?"

실습 코드: 초기화와 forward

```
import numpy as np

chars = "abc"; vocab_size = 3
hidden_size = 8
rng = np.random.RandomState(42)
Wxh = rng.randn(hidden_size, vocab_size) * 0.1
Whh = rng.randn(hidden_size, hidden_size) * 0.1
Why = rng.randn(vocab_size, hidden_size) * 0.1
bh, by = np.zeros(hidden_size), np.zeros(vocab_size)

text = "abcabcabcabc"
inputs  = [chars.index(c) for c in text[:-1]]  # a b c ... a b 개(11)
targets = [chars.index(c) for c in text[1:  ]] # b c a ... b c 개(11)

def forward(inputs, h0):
    h = h0; hs, ps, hps = [], [], []
    for idx in inputs:
        x = one_hot(idx)
        hps.append(h)                    # h_{t-1} 저장 (에dWhh 필요)
        h = np.tanh(Wxh @ x + Whh @ h + bh)
        p = softmax(Why @ h + by)
        hs.append(h); ps.append(p)
    return hs, ps, hps
# 로BPTT dWxh, dWhh, dWhy, dbh, 를dby 구해위( 유도코드 , += 누적)
# 매스텝경사하강  : P -= 0.5 * np.clip(dP, -5, 5)
```

초기 손실 12.09의 의미: 기준선

- 균등 분포의 한 위치 크로스엔트로피: $-\log(1/3) = \log 3 \approx 1.099$
- 예측 대상 11곳 $\Rightarrow$ 기준선 $11 \times \log 3 \approx 12.08$

"아무것도 모를 때"의 최대 불확실성

엔트로피 $\log 3 \times$ 시점 수. 실측 초기 손실 **12.09** 가 이 기준선과 거의 같은 것은 초기 가중치가 0 근처라 모델이 **사실상 균등 분포**를 내고 있음을 뜻한다 — "모델이 지금 **무작위 추측**을 하고 있다"는 기준선(baseline).

학습이 "잘 되고 있는가"를 말할 때 반드시 이 기준선부터 견줘야 한다.

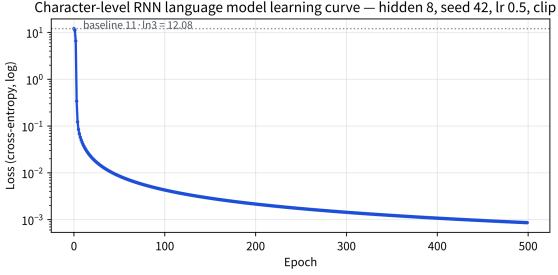
학습 결과: 14,000배 감소

500 에폭 학습 시 손실:

에폭	손실
초기	12.09 $(\approx 11 \cdot \ln 3)$
10	0.045
100	0.0043
500 (최종)	0.00086

- 손실이 기준선 12.09에서 **약 14,000배** 감소
- 학습 후 ‘’a”부터 시작해 다음 문자를 계속 예측: **“abcabcabcabc”** — 원본 패턴을 정확히 재현
- 이 작은 예제가 하는 일 = Week 17 LLM이 “다음 토큰을 예측한다”고 말하는 것의 **가장 축소된 형태**

“다음 문자 맞추기”와 “언어모델”의 차이가 크기 뿐 원리는 같다는 것을 8차원 은닉으로 확인할 수 있다.



초기 12.09 → 10에폭 0.045 → 500에폭 0.0009 (로그 스케일).

손실 곡선이 보여주는 것: 한 번에 모든 시점

- **한 번의 에폭마다 모든 시점의 예측이 동시에 개선**된다

왜 시점마다 따로 학습하면 안 되나

각 시점의 예측이 그 이전 시점들에 어떻게 의존하는지(BPTT가 계산하는 바로 그 의존성)를 전혀 반영하지 못한다. “다음 문자를 맞히는 능력”은 결국 “이전 문자들을 **얼마나 잘 요약했는가**”에 달려 — 손실을 전체 시퀀스에 대해 계산하고 역전파도 시간을 거슬러 전체에 해야 **은닉 상태가 실제로 유용한 요약을 배우게** 된다.

자주 하는 실수: 그래디언트 클리핑을 빼먹는 것

그래디언트 클리핑 단계

- **그래디언트 지수적 증가**

이 예제의 정량적 실측

학습률 2.0, 클리핑 **끄고** 500 에폭: 세 가중치 행렬의 절대값 최댓값 합 $\approx$ **47만** (467,172).
같은 설정으로 클리핑(상한 5) **켜면**: $\approx$ **83** 에 멈춤 — **약 5,600배 차이**.

자주 하는 실수: +=을 =로 쓰는 것

- **BPTT 루프에서 +=을 =로 쓰는 실수**

BPTT 코드를 읽을 때 첫째로 확인할 두 줄

(1) **그래디언트 클리핑**이 있는가?
(2) `dWhh +=` 가 `dWhh =`가 아닌가?

실전: 잘라내기 BPTT (truncated BPTT)

1. 그래디언트 곱적이 지수적으로 소실 $\Rightarrow$ **먼 과거가 거의 학습에 기여하지 못**함
2. 계산·메모리가 T에 **비례**해 증가

잘라내기 BPTT: 짧은 창으로 BPTT

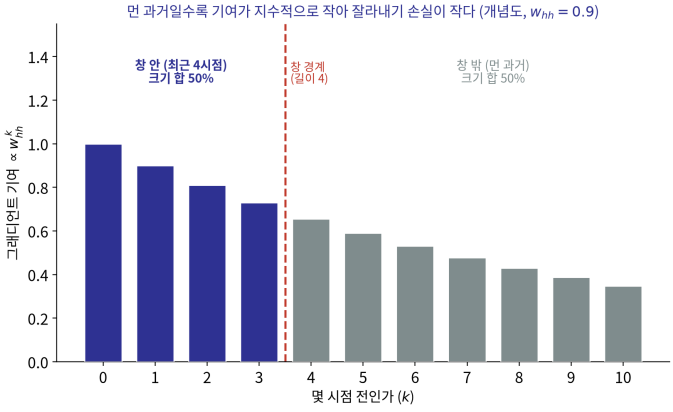
잘라내기가 “거의 같은” 이유

- 잘라내기가 “약한” 것이 아니라 “**거의 같은**” 이유: **곱적이 지수적으로 소실되기 때문**
- 300에폭 진행 시 **전체 BPTT든 창 길이 4든** “abcabcabcabc”를 정확히 재현

의의

긴 시퀀스에서 “**먼 과거의 그래디언트**”는 본질적으로 **미미**하므로 잘라내기의 **비용은 생각보다 작다**.

그림으로: 잘라내기 창이 담는 것



$w_{hh} = 0.9$ 감쇠를 가정하면(11.1절과 같은 상수), 그래디언트 기여는 몇 시점 전인지에 따라 지수적으로 작아진다 — 최근 4시점 창이 전체 기여의 상당 부분을 이미 담고 있다.

FAQ: 에폭이 많은 이유, W_{hh} 미분

Q. 왜 이렇게 많은 에폭이 필요했나요?

일부러 아주 단순한 구조(은닉 8)와 **기본 경사하강법** (Adam이 아니라)으로 **손으로 따라가기 쉽게** 만들었기 때문 — 실전에서는 Adam(Week 9의 SGD를 개선한 옵티마이저) + 더 큰 은닉 크기로 훨씬 적은 반복에 수렴. **중요한 것은 몇 에폭이 걸렸는가가 아니라, 손실이 실제로 0에 가깝게(12.09 $\rightarrow$ 0.00086) 줄었고 생성 문자열이 정답과 일치했다는 사실 자체.**

Q. W_{hh}가 공유되니까 BPTT에서 "한 번만" 미분하면 안 되나요?

아니다 — **미분은 각 시점에서 각각** 해야 하고, 그 결과를 같은 W_{hh}의 그래디언트 텐서에 **더해야** 함 (+=).

- 공유된 파라미터의 그래디언트 = 계산 그래프에서 **그 파라미터가 몇 번 쓰였는지만큼**의 기여 합
- CNN의 필터(Week 10)도 같은 원리: 하나의 필터가 이미지 위에 n^2번 쓰이면 그 필터의 그래디언트는 n^2개 위치의 **기여 합**

확인 문제: 숫자 세 개

1. **초기 손실** — 어휘 3문자의 이 언어모델에서, 무작위 초기화($\approx$ 균등 분포)의 초기 손실을 "시점 수 $\times \log V$"로 예상하라. 12문자 시퀀스(11개 예측)라면 **얼마인가**? ($11 \times \log 3$) 이 "기준선"을 아는 것이 왜 중요한지 한 문장으로.
2. **그래디언트 누적** — 손계산 예제의 $\frac{\partial L}{\partial W_{hh}} = \begin{bmatrix} 0.1000 & 0.1587 \\ -0.0365 & -0.0579 \end{bmatrix}$ 이 값이 **두 시점의 기여 합**임을 유도 식 $(A) + (B)$로 설명. `dWhh = ...`(대입)를 썼다면 어떤 값만 남았을지 추정.
3. **곱적의 양** — $\tanh'(z) \approx 0.5$, W_{hh} 크기 1.2일 때, $T = 20$이면 $\prod_{t=2}^{20}(0.5 \times 1.2)$는 **몇 배**? ($0.6^{20} \approx 3.7 \times 10^{-5}$) 소실인지 폭발인지, 왜 그런지 한 문장으로.

11.3 그래디언트 소실의 재발과 LSTM/GRU

그래디언트 소실이 시간 축에서 재발

- W_{hh} 고유값 < 1: 지수적으로 0에 가까워짐 — **그래디언트 소실**
- W_{hh} 고유값 > 1: **발산** — **그래디언트 폭발**

- **기본 RNN은 먼 과거 정보를 기억하지 못한다**

$$(0.5 \times 0.9)^{20} = 0.45^{20} \approx 4.5 \times 10^{-7} \quad (\text{사실상 } 0)$$

“소실”이 지수함수라는 것: 표로

$\frac{\partial h_T}{\partial h_1} = \prod_{t=2}^{T} tanh'(z_t)\, w_{hh}$ (스칼라 단순화)를 T를 크게 올려가며 계산:

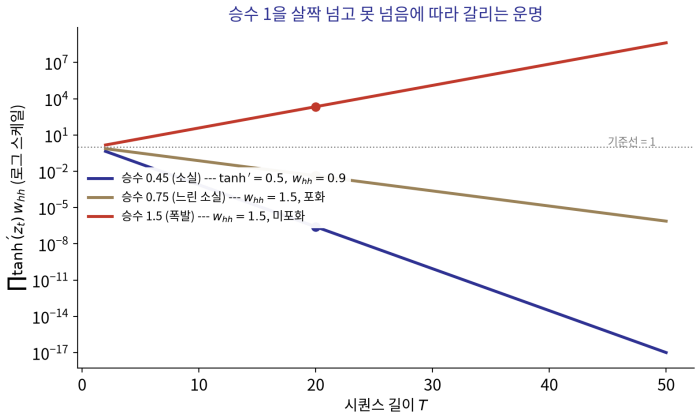
시퀀스 길이 T	tanh′ $= 0.5,\ w_{hh} = 0.9$ (승수 0.45)	$w_{hh} = 1.5$, 포화 tanh′ $= 0.5$ (승수 0.75)	$w_{hh} = 1.5$, 미포화 tanh
5	$0.45^4 \approx 4.1 \times 10^{-2}$	$0.75^4 \approx 3.2 \times 10^{-1}$	$1.5^4 \approx 5$
10	$0.45^9 \approx 7.6 \times 10^{-4}$	$0.75^9 \approx 7.5 \times 10^{-2}$	$1.5^9 \approx 3$
20	$0.45^{19} \approx 2.6 \times 10^{-7}$	$0.75^{19} \approx 4.2 \times 10^{-3}$	$1.5^{19} \approx 2.2$
50	$0.45^{49} \approx 1.0 \times 10^{-17}$	—	—
100	$0.45^{99} \approx 4.7 \times 10^{-35}$	—	—

세 열을 읽으면 RNN 학습의 “운명”

- 승수 0.45: 전형적 소실
- 승수 1.5: 폭발

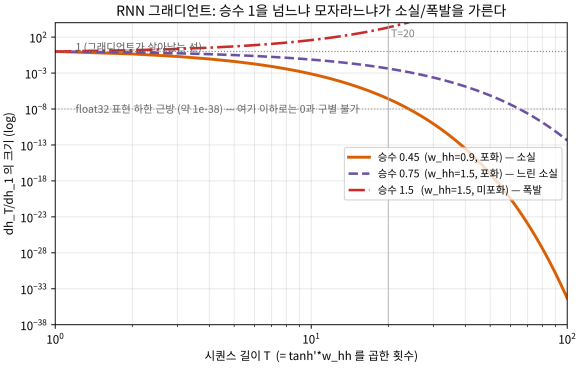
지수함수 형을 바꾸는 게이트의 임무

그림으로: 세 가지 운명



승수(교재 표의 0.45/0.75/1.5)가 1을 얼마나·어느 방향으로 벗어나는지에 따라 소실·느린 소실·폭발로 갈린다 — 로그 스케일에서 직선으로 보이는 것이 바로 “지수함수” 형이다.

소실/폭발 갈라지는 세 곡선



승수 0.45 / 0.75 / 1.5의 그래디언트 곱이 시퀀스 길이 T에 대해 지수적으로 소실 · 느린 소실 · 폭발로 갈라지는 곡선.

핵심 통찰

"**승수의 밑**이 1보다 작으면 지수 소실, 크면 지수 폭발".

- 초기화를 W_{hh} 고유값이 1 근처에 오도록 정교하게 맞춰야 한다는 11.2절의 "자주 하는 실수"가 여기서 그 **깊이**를 드러냄
- 그러나 어떤 초기화도 **지수**라는 형은 못 바꿈 ⇒ **구조**를 바꿔야 함

왜 곱셈이 문제고 덧셈은 아닌가

곱과 덧셈에 따라 그래디언트 운명이 갈린다

기본 RNN (11.2절): 이전 은닉 상태를 **곱**해서 전달

$$h_t = \tanh(\cdots + W_{hh}\, h_{t-1}) \quad \Longrightarrow \quad \frac{\partial h_t}{\partial h_{t-1}} \propto W_{hh} \ \text{(매 단계 곱)}$$

매 단계마다 그래디언트를 **할인율** < 1로 곱함.

ResNet 스킵 연결 (Week 10.2): **더함**

$$y = F(x) + x$$

$$\frac{\partial y}{\partial x} = \frac{\partial F(x)}{\partial x} + 1$$

"+1"이 매 스킵마다 정확히 1의 그래디언트를 통과시킴 — 미분 경로가 1로 "단락"해 있다. 스킵 20개를 쌓아도 0.45^{20}이 아니라 $(1 + 0.45)^{20} \approx 3.7 \times 10^2$ — **할인 없이 1 그대로** 한 가닥을 통과.

LSTM의 “고속도로”: ResNet 스킵의 시간 축 버전

- **LSTM 고속도로 = ResNet 스킵의 시간 축 버전**
- **LSTM의 셀 상태** C_t는 매 시점 “전부 버리고 새로 계산”하는 것이 **아니라**:

$$C_t = \underbrace{f_t \odot C_{t-1}}_{\text{기억 유지 (덧셈·스킵)}} + \underbrace{i_t \odot \tilde{C}_t}_{\text{신규 정보 추가 (덧셈)}}$$

- 이전 셀 상태에 **더해져서** 전달된다

게이트(gate): LSTM 3개, GRU 2개

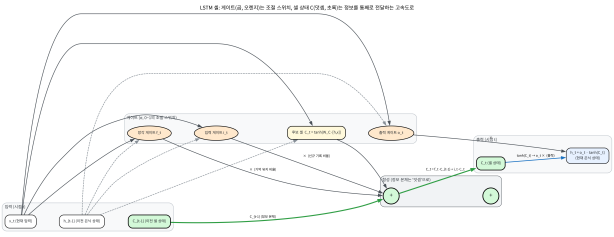
게이트는 0 1 연속 스위치

모델	게이트	역할(한 줄)
LSTM	**망각 게이트** f_t	“기억(셀 상태)에서 지금 버릴 부분은?” — 1이면 그대로 유지
LSTM	**입력 게이트** i_t	“지금 새로 기억에 쓸어 넣을 정보는?”
LSTM	**출력 게이트** o_t	“저장된 기억 중 지금 출력에 보여줄 부분은?”
GRU	**갱신 게이트** z_t	망각+입력을 하나로 통합: “기억의 얼마나를 새 값으로 교체할지”
GRU	**출력 게이트** r_t	LSTM 출력 게이트와 같은 역할

공통 분모

LSTM = 3개의 스위치, GRU = 2개의 스위치 — GRU는 LSTM의 망각·입력 게이트를 하나의 “갱신” 결정으로 **합쳐** 파라미터를 줄였다. RNN의 “은닉 상태 **전체**를 매번 새로 쓴다”를, 게이트는 “**어떤 부분은 보존하고 어떤 부분만 바꾸라**”고 학습된 판단을 내리게 만든다. 공통 원리: **곱**으로 조절하는 **스위치** + **덧**으로 전달하는 **통로**.

LSTM 셀: 곱은 스위치, 덧셈은 고속도로



이전 셀 상태 C_{t-1}이 **망각 게이트** f_t**와 곱해지고**(조절), **입력 게이트** $i_t \cdot \tilde{C}_t$**가 더해져서**(통로) C_t로 이어지는 구조.

- **곱**(게이트) = “얼마나 남길지” 조절
- **덧셈** = 정보가 **통째로** 지나가는 고속도로

이 분리(조절/통로)가 기본 RNN의 “전체 갱신(곱만)”과 구별되는 전부.

셀 상태의 그래디언트가 왜 1 근처를 유지하는가

$$C_t = \underbrace{f_t \cdot C_{t-1}}_{\text{기억 유지}} + \underbrace{i_t \cdot \tilde{C}_t}_{\text{신규 기록}} \qquad (f_t, i_t \in (0,1))$$

기억이 시점 1에서 T까지 전해지는 그래디언트: 신규 기록 항 ($i_t\tilde{C}_t$)은 C_{t-1}에 무관하므로 미분에서 **떨어져 나가고**,

$$\frac{\partial C_T}{\partial C_1} = \prod_{t=2}^{T} f_t$$

- **RNN의 $\prod tanh'(z_t)\, w_{hh}$에서 $\tanh'$ (고정 할인율, ≈ 0.45)의 자리에 학습된 망각 게이트 f_t가 들어온 것**

차이 두 가지: 1에 수렴 + 통로 오염 없음

1. **덧셈 항이 "기억 통로"를 오염시키지 않는다** — $i_t \tilde{C}_t$는 그래디언트를 **생성**하지만 기존 C_{t-1}이 통과하는 경로를 **할인하지는 않**는다

게이트가 $f_t \approx 0.99$로 학습되면 $T = 100$에서 $0.99^{99} \approx 0.37$ vs RNN $0.45^{99} \approx 4.7 \times 10^{-35}$ — **약 10^{33} 배가 잘 보존**. 승수의 **밑이 0.45에서 0.99로 이동**한 것: 아직 지수이고 1은 아니지만, **학습이 그 밑을 문제의 필요에 맞게 밀어 올릴 수 있는 것**.

게이트의 비용: 파라미터가 얼마나 늘까

모델	시점당 가중치($\approx$)	기본 RNN 대비
기본 RNN	$512 \cdot 128 + 128^2 \approx 8.2 \times 10^4$	$1\times$
GRU (게이트 3)	$3(d_{in} + d_h)\, d_h \approx 2.5 \times 10^5$	$\approx 3\times$
LSTM (게이트 4)	$4(d_{in} + d_h)\, d_h \approx 3.3 \times 10^5$	$\approx 4\times$

- 게이트 하나마다 입력$\rightarrow$은닉, 은닉$\rightarrow$은닉 가중치 한 조($(d_{in} + d_h)\, d_h$) 추가
- LSTM $\approx$ 4배, GRU $\approx$ 3배의 **메모리·계산**

역사: 1997년에 이미 알고 있었지만 생태계는 20년 뒤에 왔다

- **2013∼2014년 GPU + 프레임워크 + 초기화/옵티마이저가 한데 모여야** LSTM이 비로소 “**표준**”이 됨(2014년 seq2seq 번역·음성에서 두각)

교훈 — Week 9 “깊은 망은 세 라인(활성함수·초기화· 최적화)이 함께 풀려야 했다”와 동일

좋은 아이디어(LSTM)와 그것을 쓸 수 있는 생태계는 동시에 와야 한다.



Sepp Hochreiter

LSTM 논문 1저자 (1997, 당시 박사과정)



Jürgen Schmidhuber

지도교수, LSTM 공동 저자

Hochreiter & Schmidhuber, “Long Short-Term Memory,” Neural Computation 9(8), 1997 — 곱 대신 덧셈으로 기억을 전달하자는 해법을 이미 제안

Input sentence:	*Translation (PBMT):*	*Translation (GNMT):*	*Translation (human):*
李克強此行將啟動中加總理年度對話機制，與加拿大總理杜魯多舉行兩國總理首次年度對話。	Li Keqiang premier added this line to start the annual dialogue mechanism with the Canadian Prime Minister Trudeau two prime ministers held its first annual session.	Li Keqiang will start the annual dialogue mechanism with Prime Minister Trudeau of Canada and hold the first annual dialogue between the two premiers.	Li Keqiang will initiate the annual dialogue mechanism between premiers of China and Canada during this visit, and hold the first annual dialogue with Premier Trudeau of Canada.

Google Neural Machine Translation (GNMT) 발표, 2016.9 — 기존 PBMT(구문 기반)와 GNMT(LSTM 기반 seq2seq)의 중국어→영어 번역 비교. GNMT가 사람 번역에 훨씬 가까움

실습: “기억 과제”로 게이트가 정말 기억을 살리는가

“덧셈이 소실을 늦춘다”를 공짜로 믿지 말고 **실제로 학습해서** 확증. 과제는 의도적으로 **순차 병렬화를 배제**하고 **장기 기억만을 고립**시킨다.

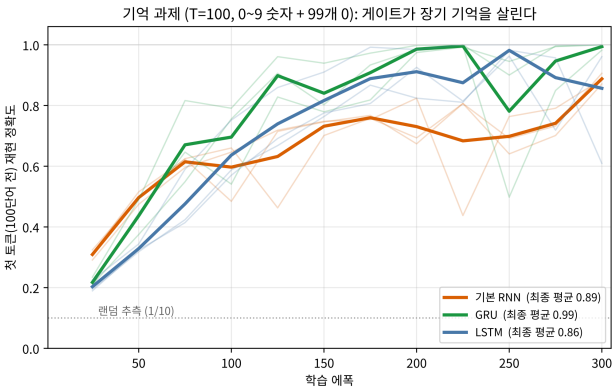
과제

길이 $T = 100$의 시퀀스를 만든다. **첫 번째 토큰 ($t = 1$)은 0∼9의 숫자 하나**(10차원 one-hot)이고, **나머지 99개 토큰은 전부 0**. 모델은 **마지막 시점 ($t = 100$)의 은닉 상태에서 “첫 번째 토큰이 뭐였는지”**를 분류.

- **중간 토큰에 새 정보가 전혀 없음** — $t = 100$의 답은 $t = 1$의 정보를 **99단계에 걸쳐 그대로 운반**할 수 있는가에 달려
- 기본 RNN: 0.45^{99} 수준의 소실로 이 “운반”을 거의 못 함
- 게이트 모델: 셀/은닉 스킵으로 정보를 **통째로 옮겨야** 함
- 세 모델, **같은 은닉 크기 32, Adam, 300에폭, 3개 seed 평균**

기억 과제: 결과

모델	seed 0	seed 1	seed 2	**평균**
기본 RNN	0.87	0.91	0.89	**0.89**
GRU	1.00	1.00	0.98	**0.99**
LSTM	0.61	1.00	0.96	**0.86**



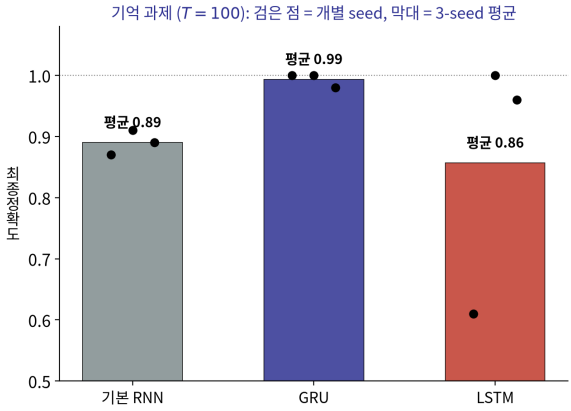
3개 모델 × 3 seed는 CPU에서 약 1~2분. (수식·코드는 실습 노트북 “기억 과제” 셀에서 그대로 실행.)

기본 RNN은 0.89에 못 미치거나 흔들리고, GRU/LSTM은 1.00에 도달.

기억 과제에서 읽어야 할 세 가지

1. 게이트가 기억을 살린다

그림으로: 기억 과제 최종 정확도



막대는 3-seed 평균, 점은 개별 seed — GRU가 가장 높고 안정적이며, LSTM은 평균은 낮지만(0.86) seed 0(0.61)의 변동이 크다: 단일 실행이 아니라 여러 seed로 판단해야 하는 이유.

LSTM/GRU: 게이트로 소실을 완화 — 원리 한 줄

게이트로 은닉 상태 선택적 유지

핵심 트릭

정보가 지나가는 경로에 **곱셈 대신 덧셈**이 섞이도록 설계 — Week 10.2에서 본 **ResNet의 스킵 연결 ($y = F(x) + x$)과 정확히 같은 원리**.
덧셈은 그래디언트를 **그대로 통과**시키므로(미분이 1), 곱셈만 반복될 때보다 **소실이 훨씬 덜**하다.

RNN의 근본적 한계: 순차성

- **RNN은 순차적으로 한 시점씩 처리**
  - **병렬화가 어려**움
  - 아주 긴 시퀀스에서는 **여전히** 먼 과거 정보가 흐려진다

다음 장으로

Week 12의 **Attention/Transformer**는 이 **순차성 자체를 없애는** 완전히 다른 접근이다 — “필요할 때마다 과거 전체를 직접 다시 들여다본다”.

그림으로: 순차성이 막는 병렬화

$t = 100$을 계산하려면 $t = 1, \ldots, 99$를 반드시 순서대로 먼저 거쳐야 한다



⇒ **병렬화 불가 --- GPU가 아무리 많아도 시점 순서를 건너뛸 수 없음**

(대비: CNN · MLP는 위치/뉴런별로 동시에 계산 가능)

게이트를 추가해도 시점 t는 $t-1$을 거쳐야만 계산된다 — 이 순서 제약이 RNN 계열 전체의 근본 한계이며, Week 12의 Attention이 없애려는 바로 그 제약이다.

실무 노트: 모델 고르는 "사다리"

기본 RNN → (길이에 따라 정확도 갈리면) **GRU** → (여전히 부족하면) **LSTM** → (길이 극단적) **Attention** (Week 12)

- **그래디언트 클리핑은 RNN 훈련의 의무**
- **망각 게이트는 1 근처로 초기화**

실무 노트: 과적합 감시

- **train ≫ val gap**가 **시퀀스 길이에 비례해 벌어진다면**:
  - 모델 크기를 **줄이**거나(GRU → RNN으로 한 칸)
  - **dropout / early stopping**을 더 쓰는 쪽이 **게이트를 더 붙이는 것보다 먼저** 시도해야 함

자주 하는 실수: "소실을 완화했다"고 긴 시퀀스를 **무조건 RNN으로**

게이트가 소실을 **느리게** 한다고 해서 **아무 길이든** LSTM/GRU가 "기억한다"고 믿는 것은 실수. 두 가지 함정:

- **아직도 지수다, 다만 밑이 1에 가까워졌을 뿐** — 수천 토큰 이상이면 **LSTM/GRU도** 먼 과거를 서서히 잃어감. "완화이지 **해결이 아니다**" — 길이가 극단적일 때는 **Attention**으로 가는 게 정답
- **과적합** — 수천 문장 이하의 작은 문제에서 "시퀀스가 짧아 소실이 안 생길 것 같은데" LSTM을 올리면, 소실은 안 생기고 **과적합만** 생김 — "소실 증상"(훈련 중에도 긴 시퀀스 정확도가 안 옴)과 "과적합 증상"(train ≫ val)을 구분할 줄 알아야 한다

FAQ: 완화인가, 해결인가, GRU는 못나나, 셀 vs 은닉

Q. LSTM/GRU를 쓰면 그래디언트 소실이 완전히 사라지나요?

아니다 — "**완화**이지 "해결"이 아니다. 게이트가 곱셈 경로 옆에 **덧셈 경로**를 추가해서 소실 속도를 크게 늦추지만, **수천 토큰 이상**이면 LSTM/GRU도 여전히 먼 과거 정보를 점점 잃어감. **Week 12 Attention이 "순차적으로 압축해서 기억한다"는 접근 자체를 버리고 "필요할 때마다 과거 전체를 직접 다시 들여다본다"로 전환하는 이유가 바로 이것.**

Q. GRU는 LSTM보다 항상 나쁜가요(게이트가 더 적으니까)?

아니다 — 파라미터가 적어(게이트 3개 대신 2개) **학습이 빠르고 데이터가 적을 때 과적합 위험도 낮음**. 성능은 문제에 따라 둘 중 어느 쪽이 나을지 갈림 — "게이트가 많을수록 무조건 좋다"는 직感は 성립하지 않음 (Week 6 편향-분산 트레이드오프. 위 "GRU 0.99 vs LSTM 0.86"이 직접 증거).

FAQ: 셀 상태 vs 은닉 상태 — LSTM의 구조적 핵심

Q. 게이트 모델의 “셀 상태”와 기본 RNN의 “은닉 상태”는 같은 것인가요?

아니다 — 이것이 LSTM의 **구조적 핵심**.

- **기본 RNN**: **하나의 벡터** h_t가 “지금의 출력에도 쓰이고 다음 시점의 입력에도 쓰이는” **둘의 역할을 동시에** 함 (**출력 = 상태**)
- **LSTM**: 이 둘을 **분리**
  - **셀 상태** C_t: 장기 기억, **덧셈으로만 전달**되어 사실상 소실되지 않는 “저장소”
  - **은닉 상태** h_t: 지금 시점의 “출력용 요약”, **셀에 게이트를 씌운 것**
- “**기억을 오래 보관한다(**C_t**)**”와 “**지금 말한다(**h_t**)**”를 **역할분담**한 것이, 소실을 **구조적으로** 줄이는 진짜 이유
- **GRU**는 이 분리를 **덜** 해(셀·은닉을 하나의 벡터로 통합) **파라미터가 줄어든다**

11주차 정리

1. **11.1 — 은닉 상태는 "감쇠하는 요약"** — 같은 세 행렬(W_{xh}, W_{hh}, W_{hy})을 T번 순서대로 돌려 파라미터를 길이와 무관하게 일정하게 만들고, w_{hh}가 "기억의 길이"를 조절 — **파라미터 공유 = CNN의 시간 축 버전**.
2. **11.2 — BPTT는 (A)+(B)의 재귀** — 본 시점 오차 + 다음 시점에서 온 그래디언트를 더하고, 공유 파라미터는 +=로 누적 — "abcabc…" 8차원 문자모델은 LLM "다음 토큰 예측"의 **가장 축소된 형태**(12.09 → 0.00086, 14,000배).
3. **11.3 — 곱은 지수, 덧셈은 통로** — 그래디언트 곱적 $\prod tanh' \, w_{hh}$가 고유값 1을 기준으로 지수 소실·폭발; LSTM/GRU는 **곱(게이트)=조절, 덧셈(셀 스킵)=통로**로 소실을 **완화** — 단, **순차성이라는 근본 한계는 남고**, 이것이 Week 12 어텐션의 계기.

다음 주 — Week 12. 어텐션과 트랜스포머

RNN이 "과거를 순차적으로 압축해서 기억"했다면, Attention은 "**필요할 때마다 과거 전체를 직접 다시 들여다본다**" — 11.1의 인코더-디코더 한계(긴 입력을 하나의 벡터로 압축하면 정보 손실)를 보완하러 등장하며, 11.3의 "완화이지 해결이 아니다"에 대한 **구조적** 답이다.